1. Gün Projenin amacı ve geliştirme ortamı

Staj projemde, kullanıcıların oyunları keşfedebileceği ve ilgilendikleri oyunları kişisel kitaplıklarına ekleyebileceği GameCodex adlı bir mobil uygulama üzerinde çalıştım. Uygulamanın kapsamını oyun arama, oyun bilgilerini görüntüleme, geliştirici stüdyoları inceleme ve medya içeriklerine erişme olarak belirledim. Oyun verileri için RAWG servisini kullandım. Bu projede ayrı bir sunucu veya oyun veritabanı geliştirmedim; mobil uygulamayı mevcut servisin sunduğu verilerle çalışacak şekilde ele aldım.

Geliştirme ortamında React Native ve TypeScript tabanlı proje yapısını inceledim. package.json dosyasındaki bağımlılıkları, uygulamayı başlatma komutlarını ve Android tarafındaki derleme dosyalarını gözden geçirdim. React Native bileşenlerinin ekranları oluşturduğunu, TypeScript tanımlarının ise verilerin beklenen biçimde kullanılmasına yardımcı olduğunu öğrendim. Android uygulamasının JavaScript koduyla birlikte yerel platform dosyalarına da ihtiyaç duyduğunu gördüm.

Çalışmaları önce temel ekranlar ve veri akışı, ardından etkileşimler, kişiselleştirme ve testler şeklinde bölümlere ayırdım. Kullanıcının bir oyunu bulup ayrıntılarına ulaşmasını temel kullanım senaryosu olarak seçtim. Böylece sonraki geliştirmeleri yalnızca görünüme göre değil, bu senaryoyu tamamlamaya katkılarına göre değerlendirebileceğim bir başlangıç oluşturdum.

package.json

```
12   "dependencies": {
13     "@react-native-async-storage/async-storage": "^3.1.1",
14     "@react-navigation/bottom-tabs": "^7.18.18",
15     "@react-navigation/material-top-tabs": "^7.6.17",
16     "@react-navigation/native": "^7.3.18",
17     "@react-navigation/native-stack": "^7.18.10",
18     "@reduxjs/toolkit": "^2.12.0",
19     "axios": "^1.20.0",
20     "babel-plugin-module-resolver": "^5.0.3",
21     "react": "19.2.3",
```

Şekil 1. Projedeki koddan kesit, 12–21. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

2. Gün Dosya yapısı ve TypeScript veri modelleri

Bugün uygulamanın dosyalarını görevlerine göre düzenleyen yapıyı ele aldım. Ekranları app/screens, ortak arayüz parçalarını app/components, ekran geçişlerini app/navigators altında değerlendirdim. API iletişimini app/api, veri tiplerini app/types, yardımcı işlemleri ise app/utils içinde toplamak, bir değişikliğin hangi bölümde yapılacağını daha kolay bulmamı sağladı. Her ekranın görünümünü ve stillerini ayrı dosyalarda tutarak uzun ekran dosyalarının okunabilirliğini artırmayı amaçladım.

Oyun verisinin uygulama içinde nasıl temsil edildiğini IGame ve IGameDetail arayüzleri üzerinden inceledim. Kimlik, ad, kapak görseli, kullanıcı puanı ve Metacritic puanı gibi alanları belirledim. Liste ekranının ihtiyaç duyduğu bilgiler ile ayrıntı ekranının ihtiyaç duyduğu bilgilerin aynı olmadığını gördüm. Bu nedenle ayrıntı modelinin ortak oyun modelini genişletmesini kullandım.

Bazı alanların her API yanıtında bulunmayabileceğini dikkate aldım. İsteğe bağlı alanların TypeScript tarafında belirtilmesi, bileşenleri yazarken eksik veri olasılığını görmemi sağladı. Bununla birlikte tip tanımlarının tek başına internetten gelen veriyi doğrulamadığını öğrendim. Liste ve kimlik kontrollerini çalışma sırasında da yapmak gerektiğini sonraki servis ve ekran çalışmalarım için not ettim.

app/types/game.ts

```
22  export interface IGame {
23      id: number;
24      slug: string;
25      name: string;
26      released: string;
27      background_image: string;
28      rating: number;
29      metacritic: number;
30      added: number;
31      playtime: number;
32      parent_platforms?: IParentPlatform[];
33      platforms?: IPlatform[];
```

Şekil 2. Projedeki koddan kesit, 22–33. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

3. Gün RAWG API bağlantısı ve servis katmanı

Bugün oyun verilerine erişmek için kullanılan servis katmanını geliştirdim. Ağ isteklerini ekranların içinde tekrar tekrar yazmak yerine, oyun işlemlerini gameService içinde topladım. Oyun listesi, oyun ayrıntısı, ekran görüntüleri ve fragmanlar için ayrı metotlar kullandım. Böylece ekran bileşenleri isteğin adresini oluşturmak yerine ihtiyaç duydukları servis metodunu çağırabilir hâle geldi.

Liste isteklerinde sayfa, sayfa boyutu, platform, tür ve sıralama gibi parametreleri ele aldım. Örneğin bir tür seçildiğinde aynı oyun listesi servisine genres parametresi gönderilmesini sağladım. Oyun ayrıntısı için ise seçilen oyunun kimliğini adresin ilgili bölümünde kullandım. JSON yanıtlarını TypeScript dönüş tipleriyle ilişkilendirerek ekranların hangi bilgileri kullanacağını belirginleştirdim.

API ile iletişimde başarılı yanıt kadar hata durumunun da önemli olduğunu gördüm. Bağlantı kesilmesi veya geçersiz bir yanıt durumunda ekranın sürekli yükleniyor görünmemesi gerektiğini belirledim. Kullanıcıya tekrar deneme imkânı sunan yapıyı servis çağrılarıyla birlikte düşündüm. Bu çalışma sonunda veri kaynağı ile arayüz arasındaki sorumlulukları ayırdım; ileride bir istek parametresi değiştiğinde bütün ekranları düzenleme ihtiyacını azalttım.

app/api/services/gameService.ts

```
54   getGameDetail: (
55       id: number,
56   ): Promise<IGameDetail> => {
57       return cachedGet<IGameDetail>(`/games/${id}`);
58   },
59   getGameDevelopmentTeam: (id: number, page: number = 1):
     Promise<IGamePersonResponse> =>
60       cachedGet<IGamePersonResponse>(`/games/${id}/development-team`, {
     page, page_size: 40 }),
61
62   searchGames: (
```

Şekil 3. Projedeki koddan kesit, 54–62. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

4. Gün Asenkron işlemler ve uygulama durumu

Bugün Redux Toolkit ile uygulamanın ortak durum yönetimini inceledim. Ağ üzerinden alınan verilerin anında hazır olmadığını göz önünde bulundurarak yüklenme, başarı ve hata durumlarını birbirinden ayırdım. createAsyncThunk ile servis çağrılarını uygulama akışına bağladım. Ekranda dispatch edilen bir işlemin sonucunu unwrap üzerinden ele almanın, try ve catch bloklarıyla daha açık bir kontrol sağladığını gördüm.

Her verinin ortak durumda saklanması gerekmediğini değerlendirdim. Favoriler birden fazla ekran tarafından kullanıldığı için ortak durumda yer aldı. Ekrana özgü yüklenme ve liste bilgilerini ise ilgili ekranın yaşam döngüsüyle birlikte yönettim. Sonraki sadeleştirmelerde, aynı API sonucunun hem ekranda hem gereksiz bir genel alanda tutulmasını azaltacak bir yaklaşım benimsedim.

Asenkron işlemlerde kullanıcı başka bir ekrana geçtikten sonra da yanıt gelebileceğini öğrendim. Bu nedenle yalnızca isteği başlatmayı değil, sonucun hâlâ geçerli olup olmadığını kontrol etmeyi de önemsedim. Hata durumunda kullanıcıya anlaşılır bir mesaj göstermek ve yüklenme durumunu sonlandırmak için kontrol akışını düzenledim. Böylece durum yönetiminin yalnızca veri saklamaktan ibaret olmadığını, ekran davranışlarını da belirlediğini kavradım.

app/redux/actions/gameActions.ts

```
25      'game/getGamesProcess',
26      async (params, { rejectWithValue }) => {
27          try {
28              return await gameService.getGames(params);
29          } catch {
30              return rejectWithValue(
31                  'Oyunlar yüklenemedi. Lütfen tekrar deneyin.',
32              );
33          }
34      },
35  );
```

Şekil 4. Projedeki koddan kesit, 25–35. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

5. Gün Ortak bileşenler ve görsel düzen

Bugün ekranlarda tekrar kullanılan arayüz parçalarını ele aldım. Oyun kartı, metin, sayfa kapsayıcısı, boş durum ve hata görünümü gibi bileşenleri ortak bir yapı içinde kullandım. Aynı kartı farklı ekranlarda yeniden oluşturmak yerine özellikler üzerinden beslemek, görünüm değişikliklerini daha az dosyada yapabilmemi sağladı. Bir bileşenin yalnızca görünümünü değil, kullanıcıya hangi bilgiyi öncelikli sunduğunu da değerlendirdim.

Oyun kartlarında kapak görseli, oyun adı ve puan gibi temel bilgileri birlikte gösterdim. Uzun adların düzeni bozmaması ve görselin kart alanına uygun yerleşmesi üzerinde çalıştım. AppText bileşeni ile başlık, açıklama ve yardımcı metinlerin biçimini ortaklaştırdım. Renkleri farklı dosyalara dağılmış sabit değerler yerine tema tanımlarından kullanmanın bakım açısından daha uygun olduğunu gördüm.

Yükleme sırasında gösterilen iskelet görünüm ile gerçekten veri bulunmaması durumunda gösterilen boş görünümü ayırdım. Böylece kullanıcı beklemesi mi yoksa farklı bir işlem yapması mı gerektiğini anlayabilir hâle geldi. Gün sonunda ortak bileşen yaklaşımının kod tekrarını azaltırken ekranlar arasında tutarlılık da sağladığını öğrendim. Bu temel, daha sonra eklenecek aydınlık ve karanlık tema desteği için de gerekli oldu.

app/theme/colors.ts

```
17   export const darkColors: ThemeColors = {
18       background: '#151515', // Ana koyu arka plan
19       surface: '#202020',    // Kartlar ve yan paneller
20       primary: '#64ffda',    // Aksan rengi (Neon yeşil/turkuaz - oyuncu
     ruhu)
21       secondary: '#94a3b8',  // Yardımcı metinler
22       text: '#FFFFFF',       // Ana metin
23       error: '#ff4646',
24       warning: '#ffb020',
25       success: '#4caf50',
26       border: '#333333',
27       black: '#000000',
28       white: '#FFFFFF',
```

Şekil 5. Projedeki koddan kesit, 17–28. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

6. Gün Ekran geçişleri ve alt gezinme çubuğu

Bugün uygulamanın ekranlar arası geçiş yapısını düzenledim. Ana sayfa, keşfet, arama, kitaplık ve ayarlar bölümlerini alt gezinme çubuğunda topladım. Her bölümün kendi ayrıntı ekranlarına geçebilmesi için yığın tabanlı gezinme yapısını kullandım. Ekranlara geçerken tüm nesneyi taşımak yerine ilgili kimlik gibi gerekli parametreleri göndermenin daha anlaşılır bir akış oluşturduğunu gördüm.

Alt çubuğun Android telefonun sistem gezinme alanının altında kalması sorununu güvenli ekran boşluklarını dikkate alarak ele aldım. useSafeAreaInsets üzerinden alınan alt boşluğu çubuğun yüksekliğine ve iç boşluğuna ekledim. Böylece geri ve ana ekran tuşlarının bulunduğu bölgeyle uygulama kontrollerinin çakışmasını azaltmayı amaçladım. Klavye açıldığında alt çubuğun davranışını da arama ekranıyla birlikte düşündüm.

Geçişlerin hızlı hissedilmesi için ilk anda bütün sekmelerin hazırlanması yerine ihtiyaç duyulduklarında oluşturulmasını kullandım. Etkin olmayan ekranların çalışma davranışını gezinme ayarlarıyla düzenledim. Alt sekme simgelerindeki animasyonları seçimin kullanıcı tarafından fark edilmesini sağlayacak şekilde ele aldım. Bu çalışma bana animasyonun tek başına performans olmadığını; ekran oluşturma ve veri yükleme maliyetlerinin ayrıca değerlendirilmesi gerektiğini gösterdi.

app/navigators/bottom-tab-stack.tsx

```
20   <Tab.Navigator
21       safeAreaInsets={{ bottom: insets.bottom }}
22       detachInactiveScreens
23       screenOptions={({ route }) => ({
24           lazy: true,
25           freezeOnBlur: true,
26           tabBarStyle: {
27               backgroundColor: colors.surface,
28               borderTopWidth: 1,
29               borderTopColor: colors.border,
30               height: 64 + insets.bottom,
31               paddingTop: 6,
32               paddingBottom: insets.bottom + 4,
```

Şekil 6. Projedeki koddan kesit, 20–32. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

7. Gün Ana sayfada oyunların listelenmesi

Bugün uygulamanın ana sayfasında oyunların gösterilmesini ele aldım. API tarafından döndürülen oyun listesini kart bileşenlerine aktararak kaydırılabilir bir görünüm oluşturdum. Çok sayıda kaydı tek seferde ekrana yerleştirmek yerine liste bileşenlerinin sunduğu yapıyı kullandım. Her oyunun benzersiz kimliğini liste elemanlarının ayırt edilmesinde değerlendirdim.

İlk yükleme, yenileme ve devam sayfası yükleme durumlarının kullanıcıya aynı biçimde gösterilmemesi gerektiğini gördüm. İlk açılışta içerik henüz yokken yüklenme görünümü kullanmak, mevcut listeye yeni oyunlar eklenirken ise listeyi korumak daha anlaşılır bir deneyim sağladı. Gelen sonucun gerçekten bir dizi olup olmadığını kontrol ettim. Aynı oyunun farklı sayfalarda tekrar gelmesi durumunda kimliğe göre tekilleştirme yaklaşımını kullandım.

Karttan ayrıntı ekranına geçişte oyunun kimliğini taşıyarak doğru içeriğin açılmasını sağladım. Görselin yüklenememesi veya puanın bulunmaması gibi olasılıkları kart düzeniyle birlikte değerlendirdim. Bu çalışma sonucunda liste ekranının yalnızca veriyi ekrana basmaktan oluşmadığını; yükleme, hata, yenileme ve seçim durumlarını birlikte yönetmesi gerektiğini öğrendim. Oluşturulan düzeni diğer oyun listelerinde de kullanabilecek şekilde ele aldım.

app/utils/paginated-list.ts

```
23      const response = await fetchPage(page + 1);
24      if (!Array.isArray(response.results)) throw new Error('Geçersiz oyun
    listesi');
25      if (!active) return;
26      const unique = new Map(state.items.map(item => [item.id, item]));
27      response.results.forEach(item => unique.set(item.id, item));
28      page += 1;
29      state = { items: [...unique.values()], loading: false, error: false,
    hasMore: Boolean(response.next) };
30  } catch {
31      if (!active) return;
```

Şekil 7. Projedeki koddan kesit, 23–31. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

8. Gün Tür platform ve sıralama filtreleri

Bugün oyunları türlerine ve platformlarına göre inceleme özelliği üzerinde çalıştım. Ana sayfada görülen seçimleri API isteğinin parametrelerine dönüştürdüm. Kullanıcı bir platform veya oyun türü seçtiğinde ilgili kimlik ya da slug bilgisinin oyun listesi isteğine aktarılmasını sağladım. Filtre değişiminde eski listenin devam sayfasından ilerlemek yerine yeni sorgunun ilk sayfasından başlanması gerektiğini belirledim.

Popülerlik ve puan sıralamalarının farklı anlamlar taşıdığını ayırdım. Popülerlik için API'nin added alanına dayanan sıralamasını, Metacritic için ilgili puan alanını kullandım. Puanı bulunmayan bir oyunun yüksek puanlı gibi değerlendirilmemesi gerektiğini göz önünde bulundurdum. Ayrıca yalnızca yüklenmiş bir sayfayı istemci tarafında sıralamanın bütün katalog için kesin bir sıralama sağlamayacağını öğrendim.

Filtreleme davranışını ortak yardımcılar ve liste yükleme yapısıyla ilişkilendirdim. Böylece tür, platform ve etiket ekranlarında benzer işlemler için farklı kurallar oluşmasını azaltmayı amaçladım. Kullanıcıya seçili filtrenin görünür olması ve sonuç bulunamadığında bunun açıkça belirtilmesi üzerinde durdum. Gün sonunda filtrelerin arayüz seçimi, istek parametresi ve listeyi yenileme adımlarından oluşan bütünlüklü bir işlem olduğunu kavradım.

app/api/services/gameService.ts

```
13  export interface GetGamesParams {
14      page: number;
15      pageSize?: number;
16      platforms?: number;
17      genres?: string;
18      tags?: string;
19      ordering?: string;
20      dates?: string;
21      metacritic?: string;
22      forceRefresh?: boolean;
23      creators?: string;
24  }
```

Şekil 8. Projedeki koddan kesit, 13–24. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

9. Gün Oyun araması ve yazma gecikmesinin yönetimi

Bugün oyun arama ekranını geliştirdim. Kullanıcının oyunun tam adını yazmak zorunda kalmadan sonuçlara ulaşmasını hedefledim. Arama servisinde metni boşluklardan temizleyerek search parametresine aktardım. Tam eşleşmeyi zorlayan seçenekleri kapatıp sonuçları popülerlik sırasıyla istedim. Bununla birlikte sonuçların RAWG arama davranışına bağlı olduğunu, her kelime parçası için aynı sonuçların garanti edilemeyeceğini dikkate aldım.

Her tuş vuruşunda ağ isteği göndermek yerine useDebounce yardımcı kancasını kullandım. Kullanıcı yazmaya devam ederse önceki zamanlayıcının temizlenmesini, kısa bir duraklama olduğunda güncel metnin işleme alınmasını sağladım. Böylece gereksiz istek sayısını azaltmayı ve yazma sırasında ekranın sürekli yüklenme durumuna geçmesini sınırlamayı amaçladım.

Arama kutusunda klavyenin oyun adlarını otomatik düzeltmesi sorununu da kullanıcı deneyimi açısından değerlendirdim. Arama boşken, sonuç bulunamadığında ve istek başarısız olduğunda gösterilecek durumları ayırdım. Geç gelen eski bir arama yanıtının yeni sorgunun sonuçlarını bozmaması gerektiğini belirledim. Bu çalışma sonunda hızlı aramanın yalnızca kısa bekleme süresiyle değil, gereksiz isteklerin ve geçersiz sonuç güncellemelerinin önlenmesiyle sağlandığını öğrendim.

app/hooks/useDebounce.ts

```
6    useEffect(() => {
7        const handler = setTimeout(() => {
8            setDebouncedValue(value);
9        }, delay);
10
11       return () => clearTimeout(handler);
12   }, [value, delay]);
13
```

Şekil 9. Projedeki koddan kesit, 6–13. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

10. Gün Yaratıcı ve geliştirici stüdyo araması

Bugün arama ekranını oyunların yanında kişiler ve geliştirici stüdyoları da kapsayacak şekilde ele aldım. Oyunlarla kişi ve şirket sonuçlarını tek listede karıştırmak yerine ayrı seçimler kullandım. Böylece kullanıcı hangi türde arama yaptığını görebildi. RAWG tarafında creators ve developers kaynaklarının farklı veri yapıları sunduğunu inceleyerek servis metotlarını ayırdım.

Kişi aramasında adın yalnızca belirli bir kısaltmasıyla sonuç alınması gibi durumları sorgu ve yanıt akışı açısından değerlendirdim. Metnin temizlenmesini ve API'ye doğru parametreyle gönderilmesini sağladım. Yeni bir sorgu geldiğinde artık gerekli olmayan isteğin iptal edilebilmesi için AbortSignal desteğini kullandım. Ekranlarda eski sonuçların yeni sorgu adına gösterilmemesi üzerinde durdum.

Bir stüdyo ya da kişi seçildiğinde ilgili ayrıntı ekranına kimliğiyle geçiş yapılmasını sağladım. Görseli bulunmayan profillerin de kullanılabilir kalması için alternatif görünüm yaklaşımını ele aldım. Oyun adı, kişi adı ve şirket adı gibi özel isimlerin dil değişiminde korunması gerektiğini not ettim. Bu çalışma, benzer görünen arama ekranlarının farklı kaynaklar için ayrı hata ve yükleme yönetimine ihtiyaç duyduğunu anlamamı sağladı.

app/api/services/entityService.ts

```
10   searchDevelopers: async (query: string, page: number, signal?:
     AbortSignal): Promise<IDeveloperResponse> => {
11       const response = await
     apiClient.get<IDeveloperResponse>('/developers', {
12           params: { search: query.trim(), page, page_size: 20 },
13           signal,
14       });
15       return response.data;
16   },
```

Şekil 10. Projedeki koddan kesit, 10–16. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

11. Gün Keşfet ekranı ve devam eden listeler

Bugün keşfet ekranındaki etiketler, yaratıcılar ve geliştirici stüdyoları üzerinde çalıştım. Kullanıcının yalnızca ilk gelen stüdyolarla sınırlı kalmaması için liste sonuna yaklaşıldığında sonraki sayfanın istenmesini ele aldım. Sayfa numarasını başarılı yanıt sonrasında ilerletmek ve API'nin next bilgisini kontrol etmek, gereksiz istekleri önlemek açısından önemliydi.

Aynı anda birden fazla devam isteği gönderilmesini engellemek için yüklenme durumunu kontrol ettim. Yeni sonuçları mevcut listeye eklerken kimliğe göre tekrar eden kayıtları ayıkladım. Etiket, yaratıcı ve geliştirici bölümlerinin birbirinden bağımsız yüklenebilmesini değerlendirdim. Bir bölümün hatasının diğer bütün içeriğin görünmesini engellememesi için bağımsız sonuç yönetimini önemsedim.

Ekran kapatıldığında veya sorgu değiştiğinde eski isteklerin yeni listeye veri eklememesi için yaşam döngüsü temizliğini ele aldım. Ortak sayfalama yardımcısındaki dispose yaklaşımıyla geçersiz bir oturumun sonuç yayımlamasını engelledim. Bu çalışma sırasında sonsuz kaydırmanın bütün veriyi baştan indirmek anlamına gelmediğini öğrendim. Amaç, kullanıcı ihtiyaç duydukça veriyi küçük parçalar hâlinde getirerek başlangıç maliyetini ve bellek kullanımını sınırlamaktı.

app/utils/paginated-list.ts

```
15   let page = 0;
16   let active = true;
17
18   const load = async (retry = false) => {
19       if (!active || state.loading || !state.hasMore || (state.error &&
     !retry)) return;
20       state = { ...state, loading: true, error: false };
21       onChange(state);
22       try {
23           const response = await fetchPage(page + 1);
24           if (!Array.isArray(response.results)) throw new Error('Geçersiz
     oyun listesi');
```

Şekil 11. Projedeki koddan kesit, 15–24. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

12. Gün Oyun ayrıntısı ve ilişkili içerikler

Bugün oyun ayrıntı ekranını, seçilen oyunun bilgilerini düzenli biçimde sunacak şekilde ele aldım. Başlık, kapak, çıkış tarihi, kullanıcı puanı ve Metacritic puanını birbirinden ayırt edilebilir alanlarda gösterdim. Kullanıcı puanının ölçeği ile Metacritic değerinin farklı olduğunu belirterek iki verinin karışmasını önlemeyi amaçladım. Geçersiz bir oyun kimliği geldiğinde isteğe devam etmek yerine açıklayıcı bir görünüm kullandım.

Açıklama, medya, başarımlar, ek içerikler ve Reddit bölümlerini sekmelerle ayırdım. Böylece bütün veriyi tek bir uzun ekranda üst üste göstermeden erişilebilir tuttum. Geliştirici, yayıncı ve platform gibi ilişkileri seçim yapılabilen alanlarla bağladım. Kullanıcı ilgili kaydı seçtiğinde o kaynağa ait ayrıntılara veya oyun listesine geçebilmesini sağladım.

API'de bir bilginin bulunmaması ile uygulamanın yükleme hatasını birbirinden ayırmaya dikkat ettim. Örneğin her oyunun fragmanı veya güncel Reddit içeriği bulunmayabileceğini değerlendirdim. Ekranın başlangıç değerinde önbellekteki ayrıntıyı kullanarak tekrar açılan bir oyunda boş bekleme görünümünü azaltmayı hedefledim. Bu çalışma, ayrıntı ekranının veri miktarı kadar bilginin gruplanması ve eksik veri durumlarının açıklanması açısından da tasarlanması gerektiğini gösterdi.

app/screens/detail-screen/index.tsx

```
51   const [detail, setDetail] = useState<IGameDetail | null>(() =>
     gameService.peekGameDetail(gameId));
52   const [loading, setLoading] = useState(() =>
     !gameService.peekGameDetail(gameId));
53   const [error, setError] = useState(false);
54   const [retryKey, setRetryKey] = useState(0);
55
```

Şekil 12. Projedeki koddan kesit, 51–55. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

13. Gün Kişisel kitaplık ve kalıcı favoriler

Bugün kullanıcıların beğendikleri oyunları kitaplıklarına ekleme özelliği üzerinde çalıştım. Oyun ayrıntısındaki kalp düğmesini toggleFavorite işlemiyle ilişkilendirdim. Oyun daha önce listede bulunuyorsa kaldırılmasını, bulunmuyorsa eklenmesini sağladım. Aynı oyunun birden fazla kez kaydedilmemesi için kimliğini temel aldım. Geçersiz kimliklerin kitaplığa eklenmesini önleyen kontrol kullandım.

Favorilerin uygulama kapatıldıktan sonra korunması için Redux durumunu redux-persist ve AsyncStorage ile ilişkilendirdim. Kalıcı saklanacak alanı yalnızca gerekli kitaplık verileriyle sınırladım. Ağ isteği sonuçları ve geçici yüklenme durumlarının kalıcı kayda taşınmamasını önemsedim. Bozuk veya eski biçimdeki kayıtların ayıklanması için favoritesOnlyState dönüşümünü ele aldım.

Kitaplık görünümünde oyunları iki sütunlu kart düzeninde sundum ve puan alanında Metacritic bilgisini kullandım. Yerel saklamanın bir sunucu hesabı olmadığını öğrendim: normal açma kapamalarda veri korunabilirken uygulama verilerinin temizlenmesi, uygulamanın kaldırılması veya cihaz değişikliği durumunda aynı kalıcılık beklenemez. Bu nedenle özelliği cihaz üzerinde çalışan bir kişisel liste olarak değerlendirdim. Gün sonunda veri saklama süresi ile bulut üzerinden eşitlemenin farklı konular olduğunu daha iyi anladım.

app/redux/slices/gameSlice.ts

```
12      toggleFavorite: (state, action: PayloadAction<IGame>) => {
13          const game = action.payload;
14          if (!Number.isInteger(game.id) || game.id <= 0) return;
15          const index = state.data.favorites.findIndex(item => item.id ===
    game.id);
16          if (index >= 0) state.data.favorites.splice(index, 1);
17          else state.data.favorites.push(game);
18      },
19  },
```

Şekil 13. Projedeki koddan kesit, 12–19. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

14. Gün Fragman oynatıcı ve tam ekran davranışı

Bugün oyun fragmanlarının oynatılması üzerinde çalıştım. RAWG tarafından sunulan video kaynaklarını react-native-video bileşeniyle ilişkilendirdim. API yanıtında birden fazla kalite bağlantısı bulunabildiği için ilk bağlantıyı doğrudan seçmek yerine kalite bilgisini değerlendiren bir seçim kullandım. Bu işlem mevcut kaynaklar arasında seçim yapar; düşük çözünürlüklü bir videoya yeni ayrıntı eklemez.

Küçük oynatıcı görünümü ile tam ekran oynatıcıyı ayrı durumlar olarak ele aldım. Oynatma, duraklatma ve ilerleme kontrollerinin dokunma olaylarıyla çakışmaması üzerinde çalıştım. Ekran yönü değişirken oyun ayrıntısının öne gelmesi sorununu gezinme ve video ekranının yaşam döngüsüyle birlikte değerlendirdim. Kullanıcı tam ekrandan çıktığında uygun ekran düzenine dönebilmesi için yön yönetimini kullandım.

Videonun yakınlaştırılmış gibi görünerek kenarlarının kesilmesi sorununda görüntü yerleşim biçimini kontrol ettim. Videonun tamamını oranını koruyarak göstermek için contain yaklaşımını kullandım; farklı ekran oranlarında siyah boşluk kalmasının kırpılmadan gösterimin doğal sonucu olduğunu öğrendim. Uygulama arka plana geçtiğinde oynatmanın duraklatılmasını da ele aldım. Gün sonunda video deneyiminin kaynak kalitesi, ekran oranı, gezinme ve dokunma yönetiminin birlikte çalışmasına bağlı olduğunu gördüm.

app/components/video-player/index.tsx

```
50   const videoUrl = Object.entries(sources || {})
51     .filter(([, value]) =>
52       typeof value === 'string' && /^https?:\/\//i.test(value.trim()),
53     )
54     .sort(([firstQuality, firstUri], [secondQuality, secondUri]) => {
55       return sourceQuality(secondQuality, String(secondUri)) -
56         sourceQuality(firstQuality, String(firstUri));
57     })
58     .map(([, value]) => (value as string).trim())[0] || '';
```

Şekil 14. Projedeki koddan kesit, 50–58. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

15. Gün Ekran görüntülerinin tam ekran incelenmesi

Bugün medya bölümündeki oyun görselleri için tam ekran görüntüleyici üzerinde çalıştım. Küçük bir görsele dokunulduğunda ilgili görselin açılmasını sağlamak amacıyla görsel listesi ve başlangıç sırası bilgilerini ayrı bir bileşene aktardım. Böylece görüntüleyici yalnızca tek bir ekrana bağlı kalmadan tekrar kullanılabilir hâle geldi.

Sağa ve sola kaydırma davranışını mevcut görselin sırasıyla ilişkilendirdim. İlk ve son görsellerde listenin dışına çıkılmaması için sınır kontrolü kullandım. İki parmak arasındaki mesafeyi hesaplayarak yakınlaştırma hareketini değerlendirdim. Yakınlaştırma ve konum değerlerini Animated üzerinden yönettim. Görsel değiştirildiğinde önceki yakınlaştırmanın yeni görsele aktarılmaması için dönüşüm değerlerini başlangıca döndürdüm.

Görsel yüklenirken gösterilecek durum ile bağlantı başarısız olduğunda gösterilecek durumu ayırdım. Ekran boyutunu sabit bir telefon ölçüsüne göre almak yerine useWindowDimensions ile güncel ölçülere bağlı tuttum. Bu çalışma sırasında yatay kaydırma, yakınlaştırma ve kapatma hareketlerinin aynı dokunmayı farklı biçimlerde yorumlayabileceğini gördüm. Hareketlerin birbirini engellememesi için mevcut yakınlaştırma durumunu ve hareket yönünü dikkate almak gerektiğini öğrendim. Görüntüleyiciyi kapatmanın kullanıcı için açık bir kontrolle erişilebilir kalmasını önemsedim.

app/components/image-viewer/index.tsx

```
65   const resetTransform = () => {
66       scaleValue.current = 1;
67       translateValue.current = { x: 0, y: 0 };
68       scale.setValue(1);
69       translateX.setValue(0);
70       translateY.setValue(0);
71   };
72
```

Şekil 15. Projedeki koddan kesit, 65–72. satırlar.

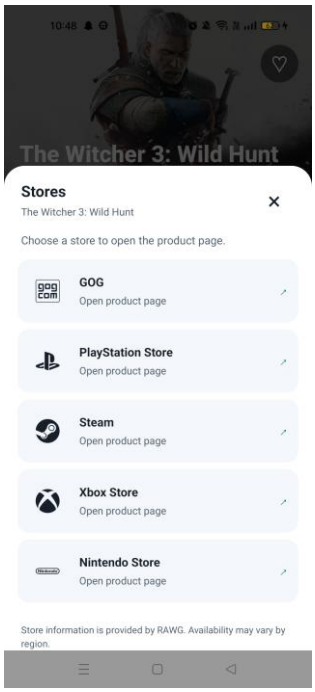
STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

16. Gün Oyun mağazaları ve satış bağlantıları

Bugün oyunların satıldığı mağazaları ayrıntı ekranına ekledim. Oyunla ilişkili mağaza bilgileri ile ürün bağlantılarını eşleştirdim. Steam, GOG, PlayStation ve Xbox gibi mağazaları logo ve adlarıyla gösteren açılır bir seçim alanı kullandım. Böylece çok sayıda mağazası bulunan oyunlarda ana ayrıntı ekranını gereksiz yere uzatmadan seçenekleri sunabildim.

Bağlantıları kullanmadan önce web adresi biçimini kontrol eden safeStoreUrl yardımcısını ele aldım. API'den gelen her metni doğrudan açmak yerine uygun HTTP veya HTTPS bağlantılarını kabul ettim. Ürün bağlantısı bulunmayan bir mağazanın varlığını göstermek ile çalışır bir satın alma bağlantısı sunmanın farklı durumlar olduğunu ayırdım. Desteklenen bağlantıların tarayıcıya veya işletim sisteminin ilişkilendirdiği uygulamaya yönlendirilmesini kullandım; her mağaza uygulamasının otomatik açılacağını varsaymadım.

Aydınlık temada beyaz mağaza simgelerinin görünürlüğünü kontrol ederek tema ile uyumlu logo görünümünü ele aldım. Aşağıdaki gerçek cihaz görüntüsünde The Witcher 3 için gelen mağaza seçenekleri görülmektedir. Bu çalışmada hem görsel sunumun hem bağlantı doğrulamasının kullanıcı deneyimini etkilediğini öğrendim.



Şekil 16. Gerçek Android cihazında mağaza seçim ekranı.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

17. Gün API anahtarı ile giriş ve oturum süresi

Bugün uygulamayı kullanacak kişinin kendi RAWG API anahtarını girebildiği başlangıç ekranını ele aldım. Bu yapı için klasik bir kullanıcı adı ve parola sunucusu kurmadım. Kullanıcının girdiği anahtarı bir API isteğiyle doğrulayıp başarılı olduğunda normal uygulama akışına geçmesini sağladım. Anahtarı olmayan kullanıcı için RAWG üzerinden anahtar edinme yönlendirmesini giriş ekranında sundum.

Anahtarın her açılışta yeniden yazılmaması için yerel kayıt ve son kullanma zamanı kullandım. Oturum süresini yedi gün olarak belirleyen sabiti saklama işlemiyle ilişkilendirdim. Uygulama açılışında kaydın biçimini ve süresini kontrol ettim. Kayıt okunamazsa veya süresi dolmuşsa giriş ekranına dönülebilecek bir akış oluşturdum. Kayıt işlemi tamamlanmadan oturumu açılmış kabul etmemeye dikkat ettim.

Ayarlar ekranına çıkış işlemi ekleyerek kullanıcının giriş ekranına tekrar erişmesini sağladım. AsyncStorage kullanımının şifreli bir anahtar kasası anlamına gelmediğini ve yerel süre kontrolünün RAWG anahtarının kendisini iptal etmediğini öğrendim. Bu nedenle özelliği cihaz içi bir oturum kolaylığı olarak değerlendirdim. Daha geniş dağıtım için anahtarın güvenli saklanmasının ayrıca geliştirilmesi gereken bir konu olduğunu not ettim.

app/services/api-key-storage.ts

```
36  async save(key: string): Promise<void> {
37      const session: StoredApiKey = {
38          key,
39          expiresAt: Date.now() + KEY_TTL_MS,
40      };
41
42      await AsyncStorage.setItem(
43          STORAGE_KEY,
44          JSON.stringify(session),
45      );
46  },
```

Şekil 17. Projedeki koddan kesit, 36–46. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

18. Gün Tema ve Türkçe İngilizce dil desteği

Bugün uygulamanın kişiselleştirme seçeneklerini geliştirdim. Aydınlık ve karanlık tema için ayrı renk tanımları kullandım. Ekranların etkin tema üzerinden renk almasını sağlayarak arka plan, metin, kart ve sınır renklerini birlikte değiştirdim. Yalnızca arka planı açmanın yeterli olmadığını; beyaz metin ve simgelerin de yeni zeminle uyumlu olması gerektiğini gördüm.

Türkçe ve İngilizce metinleri ortak çeviri yapısında ele aldım. Menü başlıkları, düğmeler ve ayar açıklamaları için dil anahtarları kullandım. Oyun türleri ve bilinen etiketler için yerelleştirme eşleştirmeleri hazırladım. Oyun ve şirket adları gibi özel isimleri korudum. Sözlükte bulunmayan bir etiketin özgün adının gösterilmesini, yanlış veya anlamsız bir çeviri üretmeye tercih ettim.

Açıklama metinlerinin çevirisini sabit arayüz metinlerinden ayrı değerlendirdim; dış çeviri hizmeti başarısız olduğunda özgün metnin erişilebilir kalmasını sağladım. Giriş ekranına dil ve tema seçimlerini ekleyerek kullanıcı oturum açmadan da tercih yapabilsin istedim. Ayarların yeniden açılışta korunması için yerel saklama kullandım. Bu çalışma sonunda yerelleştirmenin tek bir çeviri düğmesi eklemekten ibaret olmadığını, metin uzunluğu ve okunabilirliğin de kontrol edilmesini gerektirdiğini öğrendim.

app/i18n/index.ts

```
31  const dictionary: Record<Language, Record<TranslationKey, string>> = {
32      en: {
33          home: 'Home',
34          discover: 'Discover',
35          search: 'Search',
36          library: 'Library',
37          settings: 'Settings',
38          settingsTitle: 'Settings',
39          appearance: 'Appearance',
40          appearanceDescription: 'Choose how the app looks.',
41          darkMode: 'Dark',
```

Şekil 18. Projedeki koddan kesit, 31–41. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

19. Gün Performans iyileştirmeleri ve hata testleri

Bugün bekleme hissini azaltmak ve hatalara karşı davranışı iyileştirmek için veri akışını gözden geçirdim. Aynı isteğin kısa aralıklarla yeniden gönderilmesini azaltan bellek önbelleğini kullandım. Güncel önbellek süresini beş dakika, eski verinin kullanılabileceği ek aralığı otuz dakika ve kayıt sınırını seksen olarak ele aldım. Yakın zamanda alınmış verinin gösterilirken arka planda yenilenebilmesi, tekrar açılan ekranların boş beklemesini azaltmaya yardımcı oldu.

Aynı adrese yapılan eş zamanlı istekleri birleştiren bekleyen istek kaydını inceledim. Oyun kartına basılmaya başlandığında ayrıntı isteğini erkenden başlatan yaklaşımı kullandım. Buna rağmen ilk ağ isteğinin süresinin internet bağlantısına ve RAWG yanıtına bağlı kaldığını gördüm. Bu nedenle ölçüm yapmadan uygulamanın bütün işlemlerinin anlık olduğunu ileri sürmedim.

Testlerde API anahtarı kaydı, filtreler, sayfalama, kitaplık, yerelleştirme ve önbellek davranışlarını ele aldım. Süresi dolmuş kayıt, yinelenen oyun ve geç gelen yanıt gibi durumları kontrol ettim. Kullanılmayan bileşenleri ve bağımlılıkları ayıklayarak bakım yükünü azalttım. TypeScript ve kod denetimiyle birlikte otomatik testleri değerlendirdim. Bu çalışma, hız iyileştirmesinin veri doğruluğunu ve hata yönetimini koruyarak yapılması gerektiğini gösterdi.

app/api/request-cache.ts

```
18   const CACHE_TTL = 5 * 60 * 1000;
19   const STALE_TTL = 30 * 60 * 1000;
20   const MAX_CACHE_ENTRIES = 80;
21
22   const cache = new Map<string, CacheEntry>();
23   const pendingRequests = new Map<string, Promise<unknown>>();
24
```

Şekil 19. Projedeki koddan kesit, 18–24. satırlar.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..

20. Gün Uygulama simgesi ve Android test paketi

Bugün GameCodex adını ve uygulama simgesini Android paketiyle birlikte ele aldım. Seçilen simgeyi farklı ekran yoğunluklarına uygun Android kaynaklarına yerleştirdim. Uyarlanabilir simgenin ön planı ve arka planını ayrı kaynaklarla tanımladım. Dosya uzantısının tek başına görsel biçimini garanti etmediğini gördüm; PNG adı taşıyan bir dosyanın gerçek içeriğini de kontrol etmenin derleme hatalarını önleyebileceğini öğrendim.

Android tarafında release derlemesini oluşturarak APK dosyasını elde ettim ve bağlı test cihazına yükledim. Bu paketin uygulama kodunu içerdiğini, normal kullanımda geliştirme bilgisayarının açık olmasını gerektirmediğini gördüm. Buna karşılık RAWG verilerini getirmek için internet erişimi ve geçerli bir API anahtarı gerektiğini ayırdım. Cihazdaki kontrollerde temel ekranların açılmasını ve kitaplık verisinin uygulama yeniden başlatıldığında korunmasını değerlendirdim.

Oluşturulan paketi test dağıtımı kapsamında ele aldım. Mevcut imzalamanın test anahtarıyla yapıldığını, mağaza yayını için ayrı yayın imzası ve dağıtım hazırlıkları gerektiğini not ettim. Proje sonunda mobil arayüz, API kullanımı, yerel saklama, medya ve test çalışmalarının birlikte bir ürün oluşturduğunu gördüm. Daha sonraki çalışmalar için farklı cihazlarda medya ve yön değiştirme senaryolarının genişletilmesini, anahtar saklamanın güçlendirilmesini ve ağ gecikmelerinin ölçülmesini geliştirme başlıkları olarak belirledim.



Şekil 20. GameCodex uygulaması için kullanılan simge.

STAJ SORUMLUSU
Onay (İmza & Kaşe)
..... / ……. / 20…..